

Assignment - 7

For this assignment I have made two separate directories called 'src' and 'test'. The 'src' file contains the developer code and 'test' file contains the tester code.

1. **Basic Test Function** Write a pytest test function that checks whether the sum of two numbers (3 and 5) equals 8.

Developer code:

```
# 1. Basic Test Function
# Write a pytest test function that checks whether the sum of two numbers
(3 and 5) equals 8.

class Arithmetics:
    def add(self, n1, n2):
        return n1 + n2
```

Tester code:

```
import pytest
from Homework.src.homework import Arithmetics

class TestArithmetics:
    @pytest.fixture(autouse=True)
    def setup(self):
        # create an instance of your Arithmetics class
        self.arth = Arithmetics()

    def test_add(self):
        res = self.arth.add(10, 5)
        assert res == 15, 'Addition error'
```

Output:

```
"C:\Wipro Training\UnitTest\pytestTraining\venv\Scripts\python.exe" "C:/Program
Files/JetBrains/PyCharm Community Edition
2025.2.6/plugins/python-ce/helpers/pycharm/_jb_pytest_runner.py" --target
Homework/test/test_homework.py::TestArithmetics.test_add
Testing started at 1:26 AM ...
Launching pytest with arguments Homework/test/test_homework.py::TestArithmetics::test_add
--no-header --no-summary -q in C:\Wipro Training\UnitTest\pytestTraining
```

```
===== test session starts
=====
collecting ... collected 1 item
```

Homework/test/test_homework.py::TestArithmetics::test_add PASSED [100%]

===== 1 passed in 0.02s

=====

Process finished with exit code 0

2. Assertion Failure Create a pytest test that intentionally fails by asserting that "hello".upper() equals "hello".

Developer code:

```
# 2. Assertion Failure
# Create a pytest test that intentionally fails by asserting that
"hello".upper() equals "hello".

def to_uppercase(self, text: str) -> str:
    return text.upper()
```

Tester code:

```
# 2. Assertion Failure
# Create a pytest test that intentionally fails by asserting that
"hello".upper() equals "hello".

def test_uppercase_failure(self):
    # This will fail because "HELLO" != "hello"
    res = self.arth.to_uppercase('hello')
    assert res == "hello", 'Uppercase Error'
```

Output: (Intentional Failure)

Homework/test/test_homework.py::TestArithmetics::test_uppercase_failure

===== 1 failed in 0.09s

=====

FAILED [100%]

Homework\test\test_homework.py:21 (TestArithmetics.test_uppercase_failure)

'HELLO' != 'hello'

Expected :'hello'

Actual :'HELLO'

<Click to see difference>

3. Fixture Usage Define a pytest fixture that returns a list of numbers [1, 2, 3]. Write a test that uses this fixture to verify the list length is 3.

Developer code:

```
# 3. Fixture Usage
# Define a pytest fixture that returns a list of numbers [1, 2, 3]. Write a
test that uses this fixture to verify the list length is 3.

def number_list(self):
    return [1, 2, 3]
```

Tester code:

```
class TestArithmetics:
    @pytest.fixture(autouse=True)
    def setup(self):
        # create an instance of your Arithmetics class
        self.arth = Arithmetics()

# 3. Fixture Usage
# Define a pytest fixture that returns a list of numbers [1, 2, 3]. Write a
test that uses this fixture to verify the list length is 3.

def test_number_list(self):
    res = len(self.arth.number_list())
    assert res == 3, 'List length error'
```

Output:

```
===== test session starts
=====
collecting ... collected 1 item

Homework/test/test_homework.py::TestArithmetics::test_number_list PASSED [100%]

===== 1 passed in 0.02s
=====
```

4. Parameterized Test Use @pytest.mark.parametrize to test a function square(x) for inputs 2, 3, 4 and expected outputs 4, 9, 16.

Developer code:

```
# 4. Parameterized Test
# Use @pytest.mark.parametrize to test a function square(x) for inputs 2,
3, 4 and expected outputs 4, 9, 16.

def square(self, n1):
    return n1 ** 2
```

Tester code:

```
# 4. Parameterized Test
# Use @pytest.mark.parametrize to test a function square(x) for inputs 2,
3, 4 and expected outputs 4, 9, 16.

@pytest.mark.parametrize("n1, exval",
                        [(2, 4), (3, 9), (4, 16)])
def test_square(self, n1, exval):
    res = self.arth.square(n1)
    assert res == exval, 'Squaring Error'
```

Output:

```
===== test session starts
=====
collecting ... collected 3 items

Homework/test/test_homework.py::TestArithmetics::test_square[2-4] PASSED [ 33%]
Homework/test/test_homework.py::TestArithmetics::test_square[3-9] PASSED [ 66%]
Homework/test/test_homework.py::TestArithmetics::test_square[4-16] PASSED [100%]

===== 3 passed in 0.05s
=====
```

5. Exception Handling Write a pytest test that verifies a ZeroDivisionError is raised when dividing by zero using pytest.raises.

Developer code:

```
# 5. Exception Handling
# Write a pytest test that verifies a ZeroDivisionError is raised when
dividing by zero using pytest.raises.

def div(self, n1, n2):
    return n1 / n2
```

Tester code:

```
# 5. Exception Handling
# Write a pytest test that verifies a ZeroDivisionError is raised when
dividing by zero using pytest.raises.

def test_div(self):
    with pytest.raises(ZeroDivisionError):
        self.arth.div(19, 0)
```

Output:

===== test session starts

=====

collecting ... collected 1 item

Homework/test/test_homework.py::TestArithmetics::test_div PASSED [100%]

===== 1 passed in 0.02s

=====

Process finished with exit code 0

Complete code - src

```
# 1. Basic Test Function
# Write a pytest test function that checks whether the sum of two numbers
(3 and 5) equals 8.

class Arithmetics:
    def add(self, n1, n2):
        return n1 + n2

# 2. Assertion Failure
# Create a pytest test that intentionally fails by asserting that
"hello".upper() equals "hello".

    def to_uppercase(self, text: str) -> str:
        return text.upper()

# 3. Fixture Usage
# Define a pytest fixture that returns a list of numbers [1, 2, 3]. Write a
test that uses this fixture to verify the list length is 3.

    def number_list(self):
        return [1, 2, 3]

# 4. Parameterized Test
# Use @pytest.mark.parametrize to test a function square(x) for inputs 2,
3, 4 and expected outputs 4, 9, 16.

    def square(self, n1):
        return n1 ** 2

# 5. Exception Handling
# Write a pytest test that verifies a ZeroDivisionError is raised when
dividing by zero using pytest.raises.

    def div(self, n1, n2):
        return n1 / n2
```

Complete code - test.py

```
# 1. Basic Test Function
# Write a pytest test function that checks whether the sum of two numbers
(3 and 5) equals 8.

import pytest
from Homework.src.homework import Arithmetics

class TestArithmetics:
    @pytest.fixture(autouse=True)
    def setup(self):
        # create an instance of your Arithmetics class
        self.arth = Arithmetics()

    def test_add(self):
        res = self.arth.add(10, 5)
        assert res == 15, 'Addition error'

# 2. Assertion Failure
# Create a pytest test that intentionally fails by asserting that
"hello".upper() equals "hello".

    def test_uppercase_failure(self):
        # This will fail because "HELLO" != "hello"
        res = self.arth.to_uppercase('hello')
        assert res == "hello", 'Uppercase Error'

# 3. Fixture Usage
# Define a pytest fixture that returns a list of numbers [1, 2, 3]. Write a
test that uses this fixture to verify the list length is 3.

    def test_number_list(self):
        res = len(self.arth.number_list())
        assert res == 3, 'List length error'

# 4. Parameterized Test
# Use @pytest.mark.parametrize to test a function square(x) for inputs 2,
3, 4 and expected outputs 4, 9, 16.

    @pytest.mark.parametrize("n1, exval",
                             [(2, 4), (3, 9), (4, 16)])
    def test_square(self, n1, exval):
        res = self.arth.square(n1)
        assert res == exval, 'Squaring Error'

# 5. Exception Handling
```

```
# Write a pytest test that verifies a ZeroDivisionError is raised when  
dividing by zero using pytest.raises.
```

```
def test_div(self):  
    with pytest.raises(ZeroDivisionError):  
        self.arth.div(19, 0)
```